Project 1 Report

Contents

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Basis . . . 2

2 Training an MLP to predict compressive force densities on a fixed topology **3**
2.1 Initial training setup . . . 4
2.2 Adding the physics loss . . . 6
2.3 Hyperparameter tuning . . . 9

3 Conclusion **11**
3.1 Results . . . 11
3.2 Next steps . . . 21
3.2.1 Improving the training data . . . 21
3.2.2 A different loss . . . 23
3.2.3 Variable fixities . . . 24
3.2.4 Mixed-mode structures . . . 24
3.2.5 Variable topologies . . . 24
3.2.6 Other thoughts . . . 24

References **25**

1 Introduction

This project is based on *Real-time design of architectural structures with differentiable mechanics and neural networks* (Pastrana et al. 2025), published as a conference paper at ICLR 2025.

1.1 Motivation

Whether for sustainability or aesthetic considerations, designers and engineers often desire to design funicular or near-funicular structures. These are structures whose geometry allows them to carry expected loadings in pure compression and/or tension. Eliminating bending from the load path can afford significant gains in material efficiency, and the resulting forms are often architecturally interesting.

However, finding the funicular form for a given loading condition that satisfies imposed structural and architectural constraints is a difficult inverse problem. Iterative optimization methods can find accurate results, but the process of

form-finding via optimization is slow enough to preclude real-time design. Pastrana and contemporaries seek faster ways to find funicular forms such that the design process is no longer rate-limited by the speed of computation.

1.2 Basis

A structure may be represented as a pin-jointed bar system with N nodes connected by M bars. This representation is useful for funicular design because a pin-jointed structure cannot support bending. Thus, any equilibrium geometry will necessarily be funicular.

Each node experiences an external load vector $\vec{p}$. Some nodes' locations are prescribed, forming boundary conditions $\vec{b} \in \mathbb{R}^L$. Once external loads and boundary conditions are established, the locations of the free nodes are determined through form-finding.

Pastrana's approach to form-finding is the Force Density Method (FDM)—a geometrically linear static equilibrium solve (Schek 1974). In FDM, each of the M bars is assigned a force density q $[F/L]$, creating a force-density vector $\vec{q} \in \mathbb{R}^M$. Force densities are analogous to axial bar stiffnesses, but their connection to geometry is perhaps more intuitive. The force carried in bar i, which has length l_i, is $q_i l_i$. Prescribing (nonzero) force densities to the bars completely determines the geometry under a specified loading condition and the equations of static equilibrium. Let $\mathcal{N}(i)$ represent the set of nodes connected to node i; q_{ij} represent the force density of element ij; and $\vec{x}_i$ represent the location of node i. Then we can define a residual force vector at each node,

$$\boldsymbol{r}_i(\boldsymbol{X}; \vec{q}) = \sum_{j \in \mathcal{N}(i)} q_{ij}(\vec{x}_i - \vec{x}_j) - \vec{p}_i(\vec{x}_i). \tag{1}$$

This is the difference between the external force applied at the node and the forces applied by the bars coming into the node, where positive force densities represent bars in compression. For an equilibrium $\boldsymbol{X}$, $\boldsymbol{r}_i \in \mathbb{R}^3$ will be zero at all free nodes.

Given a set of force densities, Schek describes an exact solution for the equilibrium geometry $\boldsymbol{X}$. Let $\boldsymbol{C}_{[M \times N]}$ be the branch-node matrix, where row i describes edge i. If edge i connects nodes j and k, then $C[i, j] = -1$ and $C[i, k] = 1$. (Edge directions are irrelevant for this method.) Let n correspond to free nodes and f to fixed nodes. $\boldsymbol{C}_{n_{[M \times n]}}$ and $\boldsymbol{C}_{f_{[M \times f]}}$, then, are sub-matrices with free and fixed nodal columns, respectively, selected from $\boldsymbol{C}$. Let $\boldsymbol{X}_{n_{[n \times 3]}}$ represent the unknown locations of the free nodes, $\boldsymbol{X}_{f_{[f \times 3]}}$ represent the known locations of the fixed nodes, and $\boldsymbol{p}_{[n \times 3]}$ represent the loads applied to the free nodes. Finally, let $\boldsymbol{Q}$ be the diagonal force-density matrix—a square matrix with the force densities $\vec{q}$ on the main diagonal.

The edge vector expression $(\vec{x}_i - \vec{x}_j)$ from eq. (1) can be expressed in matrix form as

$$\boldsymbol{L} = \boldsymbol{C}\boldsymbol{X} = \boldsymbol{C}_n \boldsymbol{X}_n + \boldsymbol{C}_f \boldsymbol{X}_f,$$

separated into known and unknown quantities. The forces $\boldsymbol{s}$ in the members satisfy $\boldsymbol{s} = \boldsymbol{Q}\boldsymbol{L}$. We left-multiply by $\boldsymbol{C}_n^T$ to sum over free nodes. Thus the force-equilibrium equation for the system is

$$\boldsymbol{C}_n^T \boldsymbol{Q} \boldsymbol{L} + \boldsymbol{p} = 0,$$

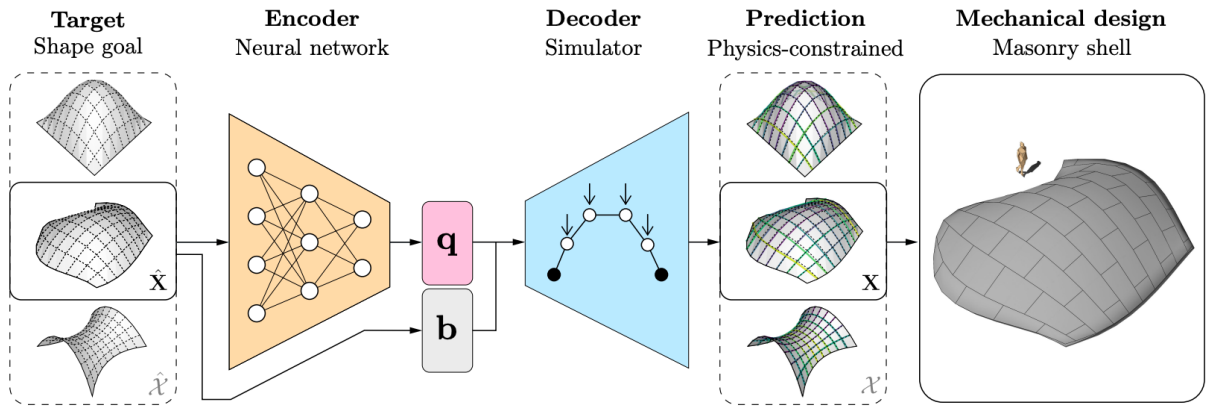

Figure 1: Architecture of the model proposed by Pastrana et al. A neural network maps a target shape $\hat{\boldsymbol{X}}$ to the stiffness space $\boldsymbol{q}$. The stiffnesses and boundary conditions $\vec{b}$ are used to find the corresponding physics-constrained shape $\boldsymbol{X}$. Figure from (Pastrana et al. 2025).

which expands to

$$\boldsymbol{C}_n^T \boldsymbol{Q} \boldsymbol{C}_n \boldsymbol{X}_n + \boldsymbol{C}_n^T \boldsymbol{Q} \boldsymbol{C}_f \boldsymbol{X}_f + \boldsymbol{p} = 0. \tag{2}$$

We often define intermediates

$$\boldsymbol{D}_n = \boldsymbol{C}_n^T \boldsymbol{Q} \boldsymbol{C_n}, \qquad \boldsymbol{D}_f = \boldsymbol{C}_n^T \boldsymbol{Q} \boldsymbol{C_f}$$

as these values will not change when evaluating multiple loading conditions on the same topology.

So, once $\vec{q}$ is determined, finding the funicular geometry under specified boundary conditions and loading reduces to solving eq. (2) for $\boldsymbol{X}_n$. However, finding the specific force densities that will produce a desired geometry remains a difficult, nonlinear inverse problem. Pastrana et al. trained a neural network to predict force densities $\hat{\boldsymbol{q}}$, given a target shape $\hat{\boldsymbol{X}}$, to produce an equilibrium shape $\boldsymbol{X}$ which is as close as possible to $\hat{\boldsymbol{X}}$. Their model architecture is described in fig. 1.

2 Training an MLP to predict compressive force densities on a fixed topology

This project aims to recreate the results of Pastrana et al. Initial fundamental differences between the implementations include that this project will implement a physics loss, and that my implementation will use PyTorch rather than JAX.

For feasibility, I approach a limited version of the full problem. When form-finding using a mesh network, variables include not just the loading conditions and force densities but also the network topology—number, fixity, location, and connectivity of nodes. An MLP could potentially support variable input lengths through padding the input vector, but to simplify the implementation, this project focuses on training a network to predict force densities to approximate the target shape on a fixed, rectangular-grid network. I believe Pastrana et al. also used this simplification.

Furthermore, like Pastrana et al., I restricted the design space to pure-compression structures (positive force densities). Complications arise with mixed-mode structures because the force densities pass through zero, but zero force densities

lead to singularity in the FDM solve.

I added a third simplification: training on only funicular targets. In real applications, the target geometry is often non-funicular. This is one reason that optimization (rather than a simple least-squares solve) is used to find force densities for a funicular shape that closely approximates the target shape. However, training on non-funicular targets makes quantifying training efficacy more difficult because I would need to compare the neural network's residuals against the residuals from another solver that would also need to be calibrated. With funicular targets, the neural network should theoretically be able to reach a loss of zero, meaning I have a consistent baseline.

2.1 Initial training setup

As previously described, on a fixed topology, equilibrium geometry is fully determined by the force densities $\vec{q}$ and applied loads $\boldsymbol{p}$. I store topology information in objects of the `MeshData` class. Each object exposes the following properties:

- `n_nodes, n_free`: number of nodes, number of free nodes
- `n_edges`: number of edges
- `free_ind, fixed_ind`: lists of the indices of free and fixed nodes
- `is_fixed`: incidence vector where 0 indicates free, 1 indicates fixed
- `C, Cf, Cn`: branch-node connectivity matrix
- `X, Xf`: original/fixed node locations
- `CnT, CfXf`: precomputed FDM terms

Upon initialization, the topology is constructed as an $n \times n$ grid. The object method `eq_shape_batched` returns equilibrium shapes for input force densities and loading matrices.

A `FixedTopologySampleGenerator` object generates training samples for a single `MeshData` object. I select force densities and loads from a uniform random distribution. Force densities range between 0.5 and 30 kN/m, and loading is constrained to the negative z direction, with magnitudes between 0 and 4 kN. Each node in the structure has a feature vector of seven properties: goal x-location, y-location, and z-location; whether the node is fixed; and the x-, y-, and z-components of the external loading. It is evident that this is more features than are necessary to determine the simplified problem, but I include them in anticipation of expanding the problem space in the future. For the initial setup on a 10 by 10 grid, this means the neural network input is of dimension 700.

I trained a neural network with two hidden layers of size 256 and $\tanh$ activation. The network class carries its own normalization mean and standard deviation, which are set by the first training sample; all other inputs are normalized by the same scale before being passed through the network to predict force densities. The output is passed through a sigmoid and scaled to the feasible output range. I set this range to $[0.05, 40]$ so the network could reach force densities near the edges of the training range without having to yield huge activations.

I trained this initial model with 2,000 maximum epochs, Adam optimizer, initial learning rate `5e-3`, early stopping patience 100, a `CosineAnnealingLR` scheduler with `T_max` 2000 and `eta_min` $1e-6$, and only a data loss. The data loss is the shape error $\mathcal{L}_{shape} = \frac{1}{N}||\hat{\boldsymbol{X}} - \boldsymbol{X}||_2$—the average Euclidean distance between the goal nodal position and the positions corresponding to the predicted force densities (only on free nodes). Generating new training data is cheap, so rather than generating training data in advance, I pass in a sampler to my training function

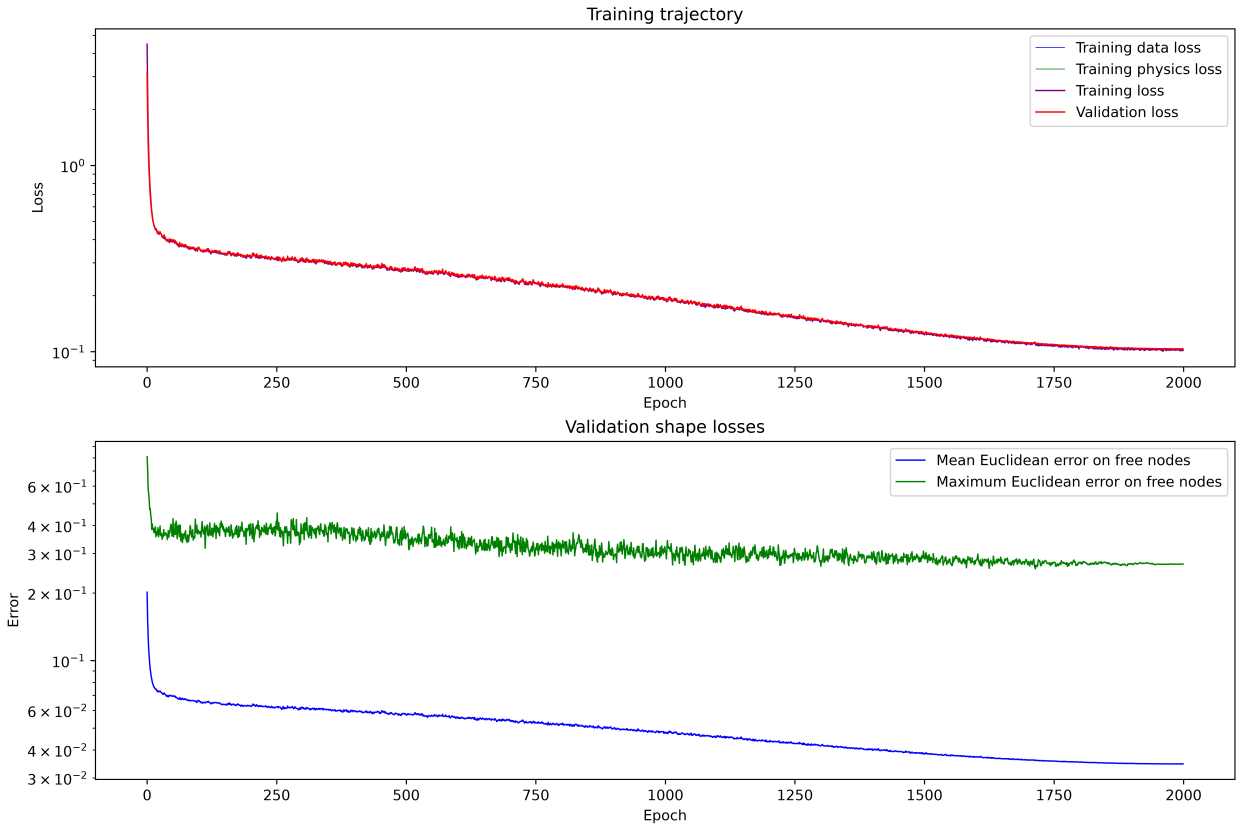

Figure 2: Top, evolution of training and validation loss across epochs when training the (256, 256) neural network on funicular targets with only data loss. Bottom, evolution of shape error.

and periodically regenerate the training data. For this run, I regenerated the data every epoch, which eliminated overfitting. Each epoch ran 10 batches, with 256 samples per batch.

Training did not stop early and took 2 minutes and 57 seconds to run. At the end of training, the training loss was 0.1016 and the validation loss was 0.1036. The training trajectory can be seen in fig. 2. The training loss and validation loss consistently show no overfitting. However, though the mean Euclidean error on the free nodes decreases proportional to the loss (as should be expected, since they're the same metric), the maximum Euclidean error on the validation set plateaus around 3×10^{-1}, or 0.3 meters. On the test set, the mean distance error is 0.0345 meters and the maximum distance error is 0.3610 meters.

Figure 3 shows the validation sample with the greatest absolute shape error. The model's prediction is in red while the ground truth is in black and blue; the worst prediction node is highlighted in orange. Figure 4 shows a sample of 9 validation samples, their ground truth values, and the model prediction.

The plots in fig. 5 attempt to find correlation between q and $\hat{q}$—for example, if predictions are worse for higher or for lower q-values. The plots are split into learnable edges: those with at least one free node whose position will propagate a gradient; and unlearnable edges: edges connected to two fixed nodes, whose force density cannot be learned via the shape loss. I left these edges in the training data because I imagine eventually training the model on meshes of varying fixity, in which the set of unlearnable edges will be different for each sample.

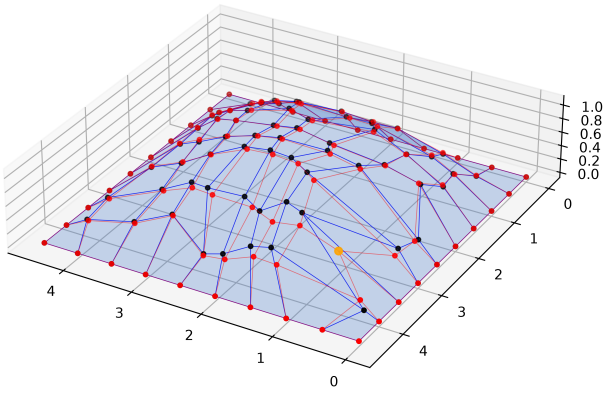

Figure 3: Ground truth versus model prediction for the sample with the highest max error in validation set. Ground truth is in black and blue; model prediction is in red.

The plots show evidence of regression toward the mean. The linear regression on the errors on learnable edges has a slope of -0.2494, and on the plot of $\hat{q}$ against q, we can compare against the $\hat{q} = q$ line to see that low force densities are overestimated and high force densities are underestimated. The plot on the right shows that the relative error is highest for low force densities. The green clouds plotted in the back show the corresponding relationships for the unknowable edges, and we see that the predictions are essentially random. This is especially clear from the middle plot; no matter the true value of q, the predictions $\hat{q}$ are symmetrically scattered around 20, which is corresponds to an output activation of zero.

2.2 Adding the physics loss

Given the residual force vector defined in eq. (1), the physics loss is the mean squared residual force across the free nodes. If the model's predicted force densities are $\hat{\boldsymbol{Q}} = \boldsymbol{Q} + \tilde{\boldsymbol{Q}}$, the real force densities plus some error, then each the physics residual for each sample is

$$\begin{aligned} \boldsymbol{r}(\boldsymbol{X}; \hat{\boldsymbol{Q}}) &= \boldsymbol{C}_n^T(\boldsymbol{Q} + \tilde{\boldsymbol{Q}})\boldsymbol{C}\boldsymbol{X} + \boldsymbol{p} && (3) \\ &= \boldsymbol{C}_n^T\boldsymbol{Q}\boldsymbol{C}\boldsymbol{X} + \boldsymbol{p} + \boldsymbol{C}_n^T\tilde{\boldsymbol{Q}}\boldsymbol{C}\boldsymbol{X} && (4) \\ &= \boldsymbol{C}_n^T\tilde{\boldsymbol{Q}}\boldsymbol{C}\boldsymbol{X}. && (5) \end{aligned}$$

Perhaps both this and what I have been calling the data loss could be considered physics losses, as they both focus on the governing equilibrium equation rather than checking the vector distance between $\vec{q}$ and $\hat{q}$. However, their interpretations are subtly different. The data loss checks the distance between the target shape and the equilibrium shape corresponding to the predicted force densities. The physics loss aggregates the unbalanced forces that would be induced at each node if the predicted force densities were prescribed to the target shape. Because of this, the physics residual attributes roughly equal precedence to all errors (which are then squared), while in contrast, the data loss heavily penalizes errors in low-q areas where changes in force density cause larger geometric shifts.

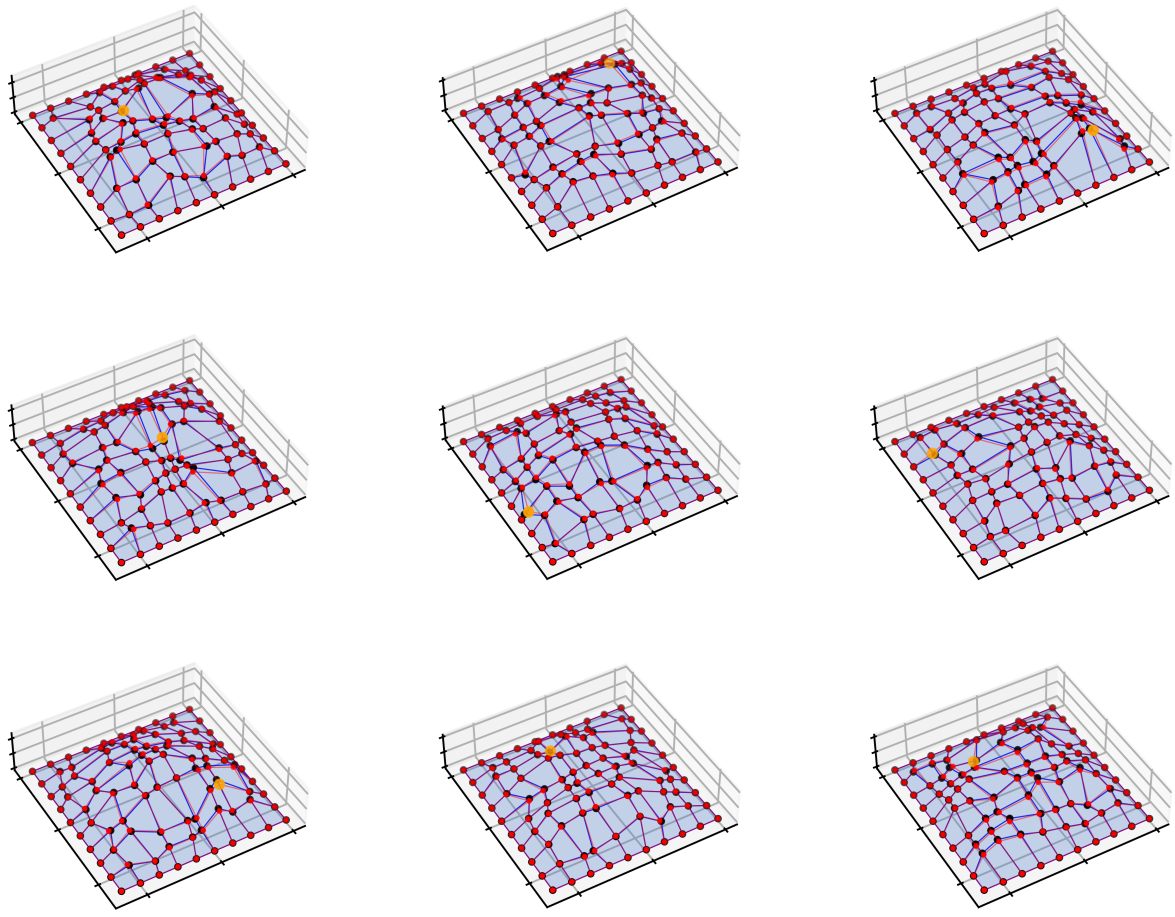

Figure 4: Ground truths and model predictions for 9 validation samples. Worst error nodes are highlighted in orange.

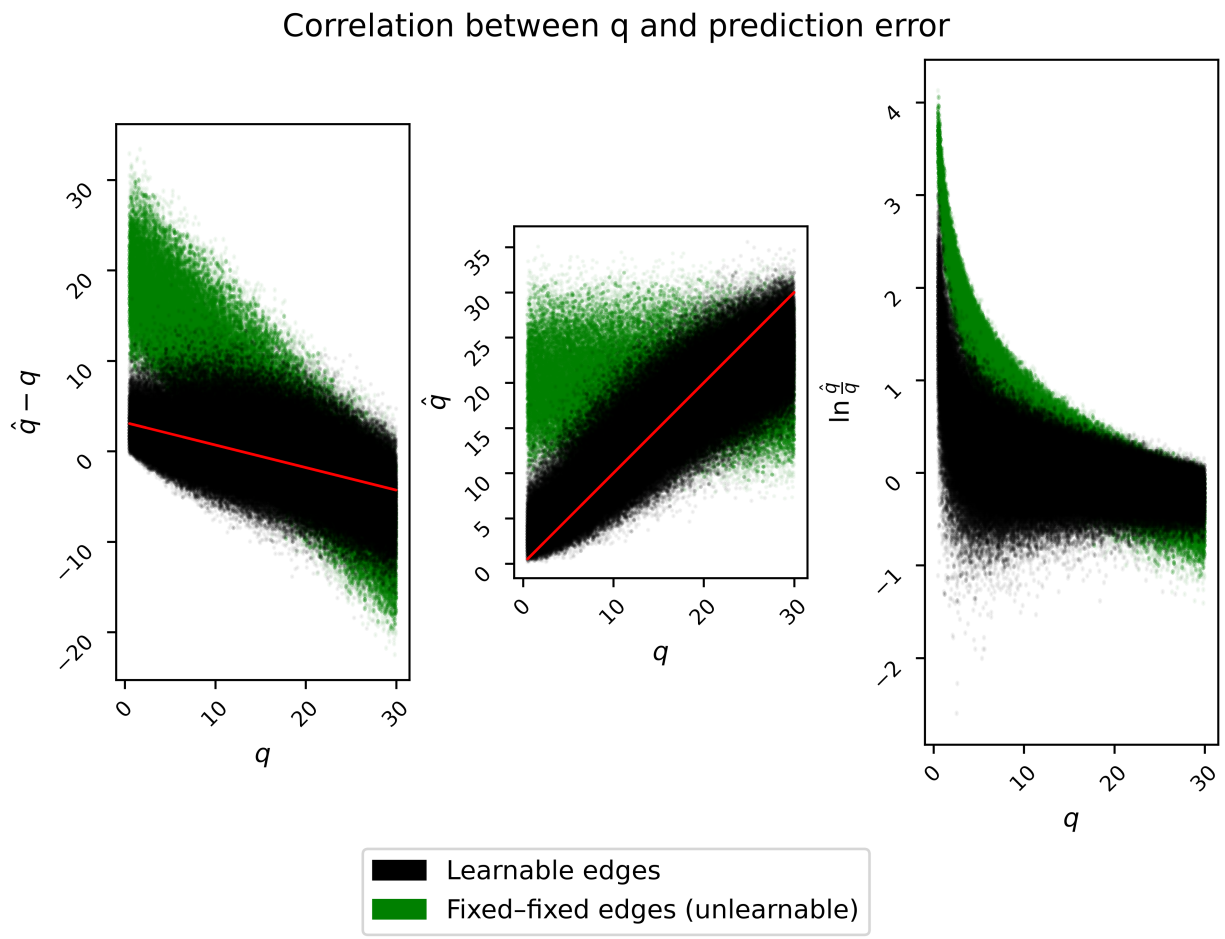

Figure 5: Plots showing the relationship between predicted and true force densities for a neural network trained on shape loss. Left, error is plotted against true q. Linear regression line is plotted in red. Center, predicted $\hat{q}$ is plotted against q. $\hat{q} = q$ is plotted in red. Right, relative error is plotted against true q. Black cloud shows results for learnable edges; green cloud shows results for unlearnable edges.

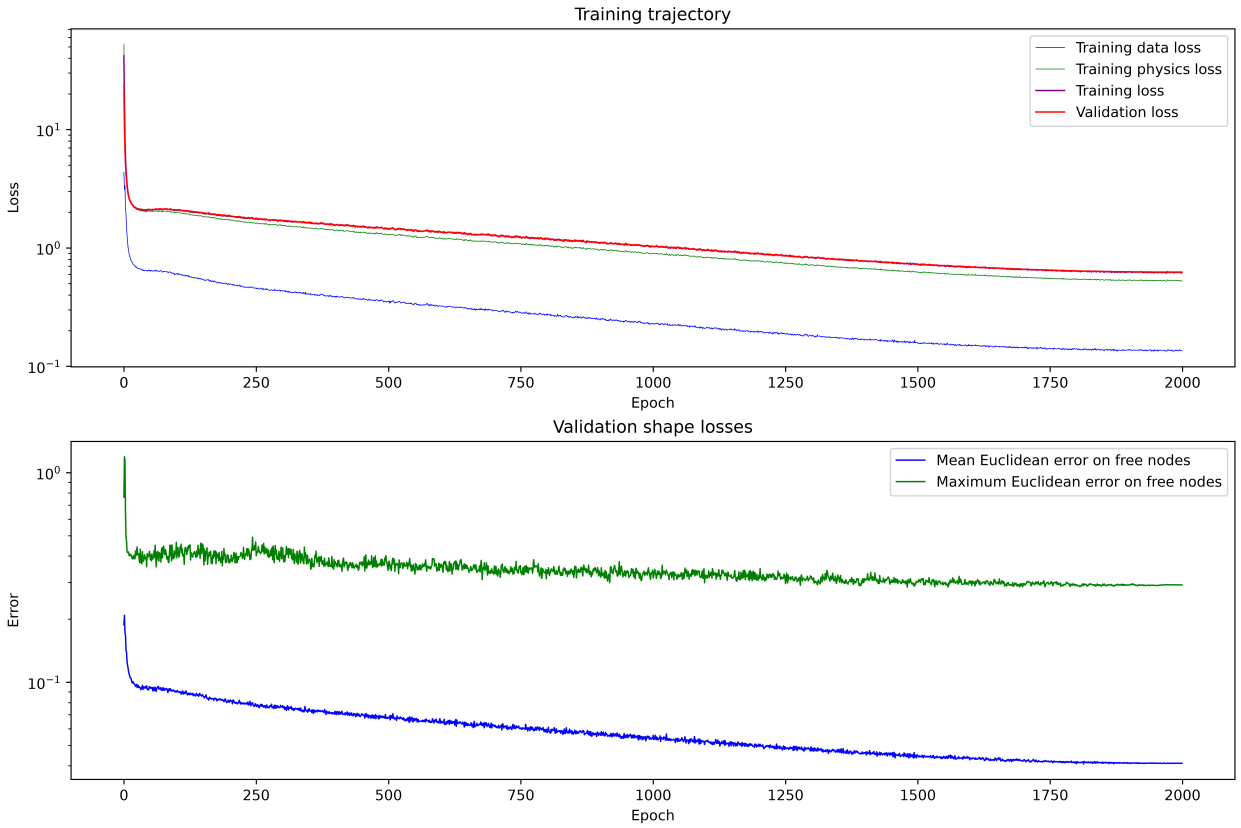

Figure 6: Training trajectory with physics loss.

I trained the neural network with the same parameters but an added physics loss and sigmoid adaptive weights. Figures 6 to 8 show the training trajectory and some sample predictions from the model trained with the physics loss.

For funicular targets, there were no tangible gains to adding the physics loss. Training took approximately 50% longer. The error-q plots look perceptually the same, with slightly higher relative errors for small values of q. The mean nodal error on the test set was 0.0410, and the maximum nodal error was 0.3898. These metrics are essentially the same, and slightly greater than, the model accuracy when trained without the physics loss. It seems the data and physics losses fought rather than cooperating. Perhaps for non-funicular targets the physics loss would be more helpful.

2.3 Hyperparameter tuning

Under both loss regimes, training ended without the loss reaching zero. In this section, I tested some hyperparameters to try to assemble a model architecture that learns more effectively. This testing was done without the physics loss enabled for efficiency. Each model was trained for 300 epochs, with Cosine annealing scheduling, with the cycle ending at epoch 300. The variables of interest were:

- Activation: Tanh, SiLU, and GeLU
- Width: 128, 256, 512, and 1024 nodes per hidden layer

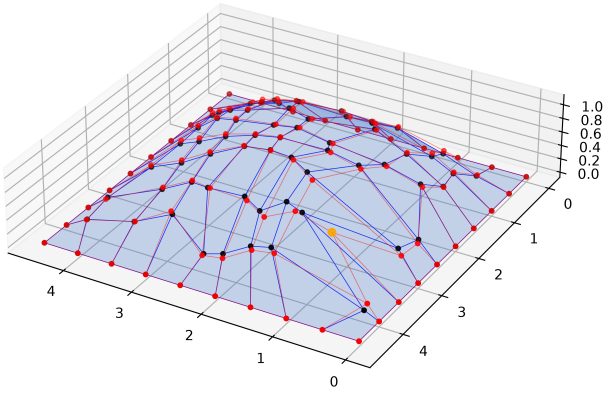

Figure 7: Worst test sample for model trained with physics loss.

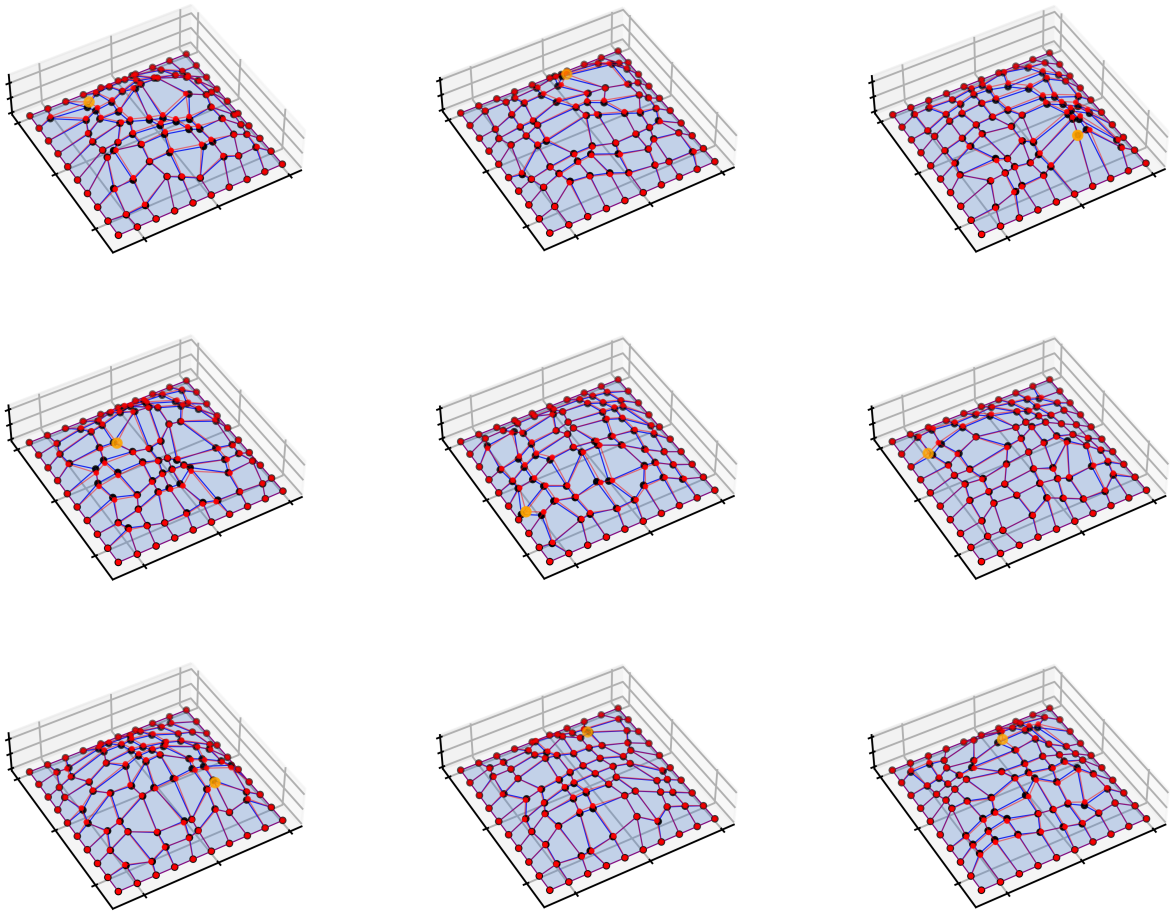

Figure 8: Sample test predictions for model trained with physics loss.

- Depth: 2, 3, 4, and 6 hidden layers
- Shape norm used for data loss: $\mathcal{L}_1$, $\mathcal{L}_{1.5}$, and $\mathcal{L}_2$.

Results are displayed in figs. 9 to 11. It is hard to determine conclusive results from the experiment because the deeper models are likely to need more epochs to learn, but the shape loss trajectories imply that the wider models perform better.

Figure 12 plots the mean shape error versus the max shape error on the test set for all of the trial models. The relationship is nonlinear, with several models achieving very low mean shape errors but high maximum shape errors. Furthermore, none of the models achieved a maximum shape error less than 0.3. The best-performing models include a mix of SiLU and GELU activations, $\mathcal{L}_{1.5}$ and $\mathcal{L}_2$ norms, and varying model sizes.

I restricted the test model set to GELU and SiLU-activated models, widths of 512 and 1024, depths of 4 and 6, and data shape norms of 1.5 and 2. I trained these 16 models for 5,000 epochs each, with results plotted in figs. 13 to 15. I selected the best model to be the 4×1024 SiLU-activated model, trained on the $\mathcal{L}_2$ shape norm. The mean nodal error on the test set was 0.016 meters, with a max test error of 0.16 meters. The 6×1024 version of the same model had worse performance after 5,000 epochs—0.018 and 0.22 meters, respectively—though I hesitate to attribute this to anything more than the larger model learning more slowly.

3 Conclusion

3.1 Results

A neural network trained on shape loss on a fixed topology is moderately successful at predicting force densities to produce a desired funicular shape under varying load conditions. I trained the 4×1024 SiLU model for 100,000 epochs, which took 5 hours and 22 minutes. At the end of training, the training loss reached 0.0035 and the validation loss was 0.0036. The training curve can be seen in fig. 16.

The spikes in the training curve are potentially caused by large gradients, which occur when the model predicts very small, incorrect force densities. I added gradient clipping, but I likely would need to lower the tolerance to eliminate the spikes.

This model had the best performance of all models so far. The average nodal error on the test set was 0.006 meters, or 6 millimeters, potentially within construction tolerance. The maximum nodal error, though, was 0.117 meters, or 11.7 centimeters. The model's predictions on samples from the test set are shown in figs. 18 and 19. The prediction error plot, fig. 17, for this model also shows better performance. The slope of the linear regression when plotting error against q is only -0.0445, which indicates less regression toward the mean. This can also be seen in the $\hat{q}$ vs q plot, where the black band fits more accurately to $\hat{q} = q$. The learnable bands are thinner, which indicates smaller errors overall. However, the increase in relative error for small q is even more pronounced.

Since the model only predicts force densities, the resulting predicted geometry will always be viable for funicular design—it just may be far from the desired geometry. The model's predictions could certainly be used as an optimization warm-start and would likely provide significant speed-ups.

Because average nodal error decreased consistently as I increased the training period (I also tried 20,000 and 50,000

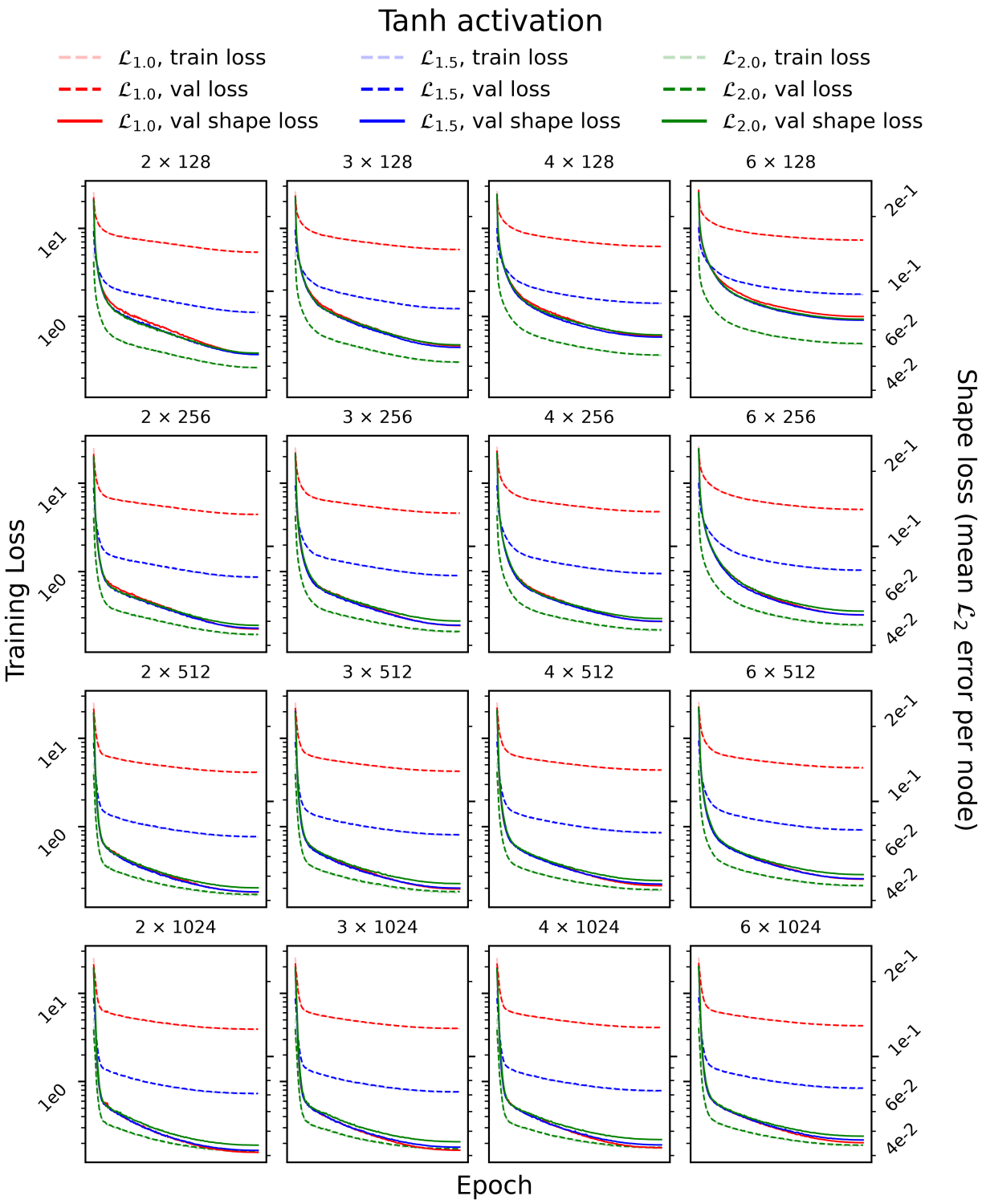

Figure 9: Tanh model performance.

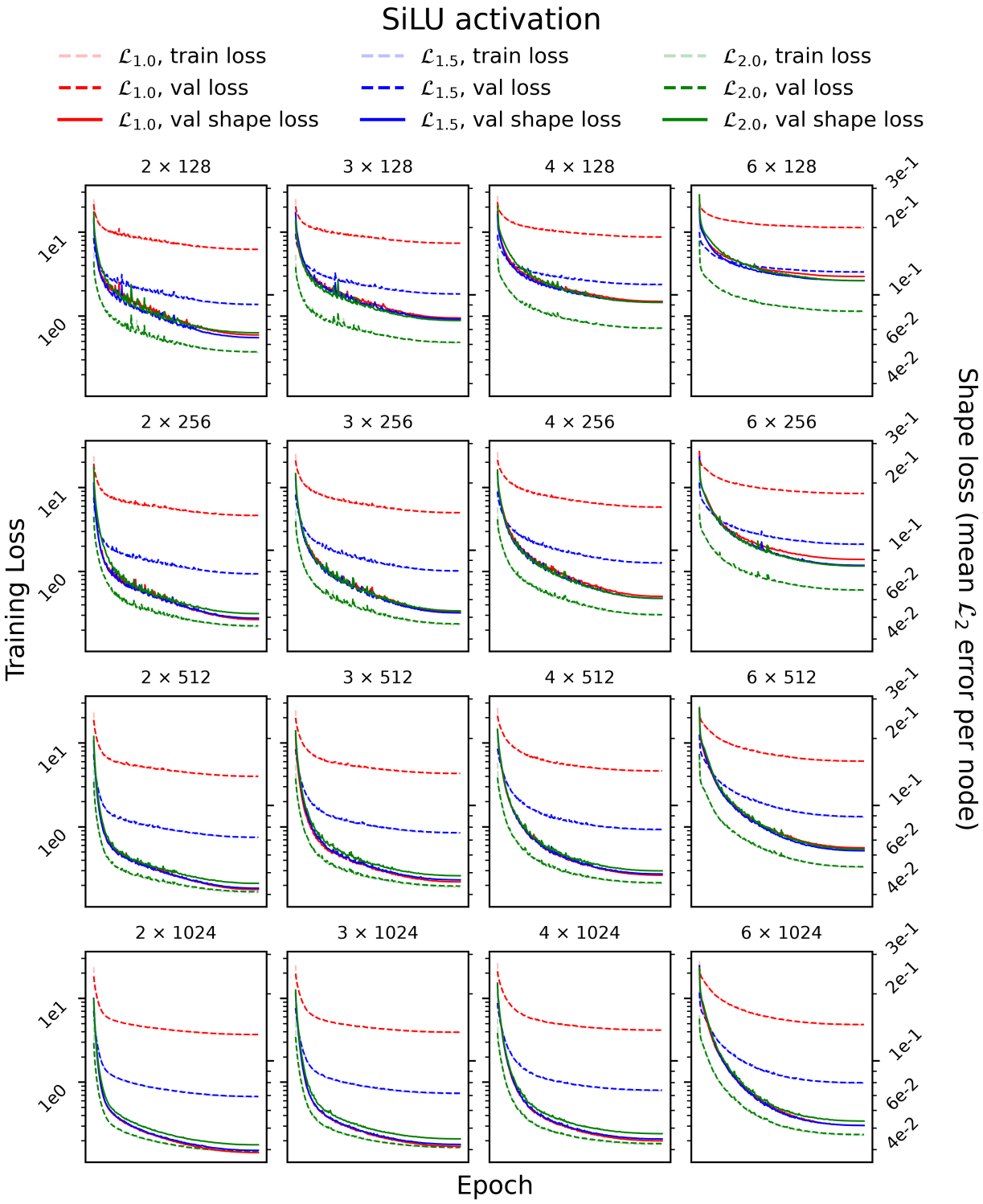

Figure 10: SiLU model performance.

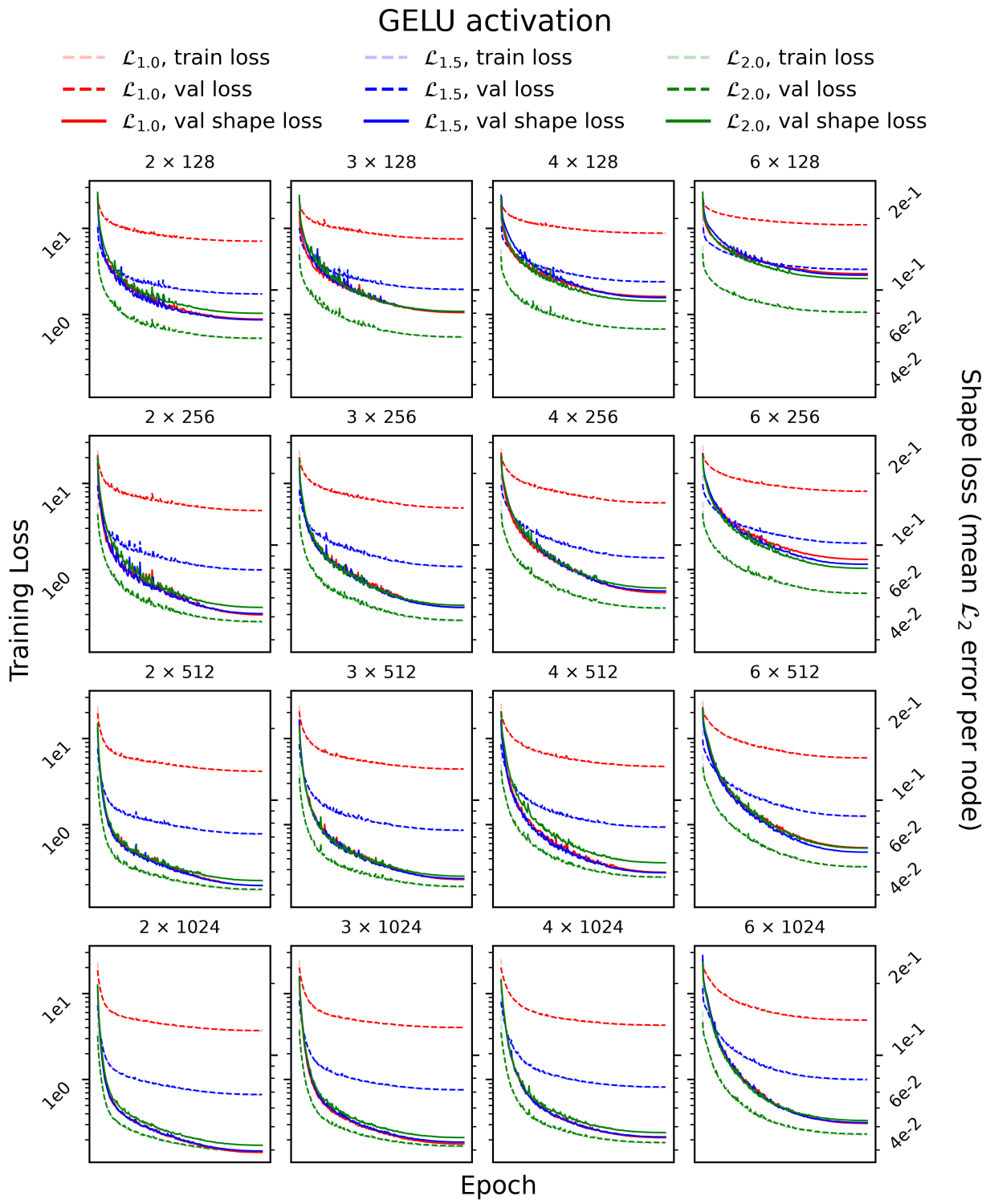

Figure 11: GELU model performance.

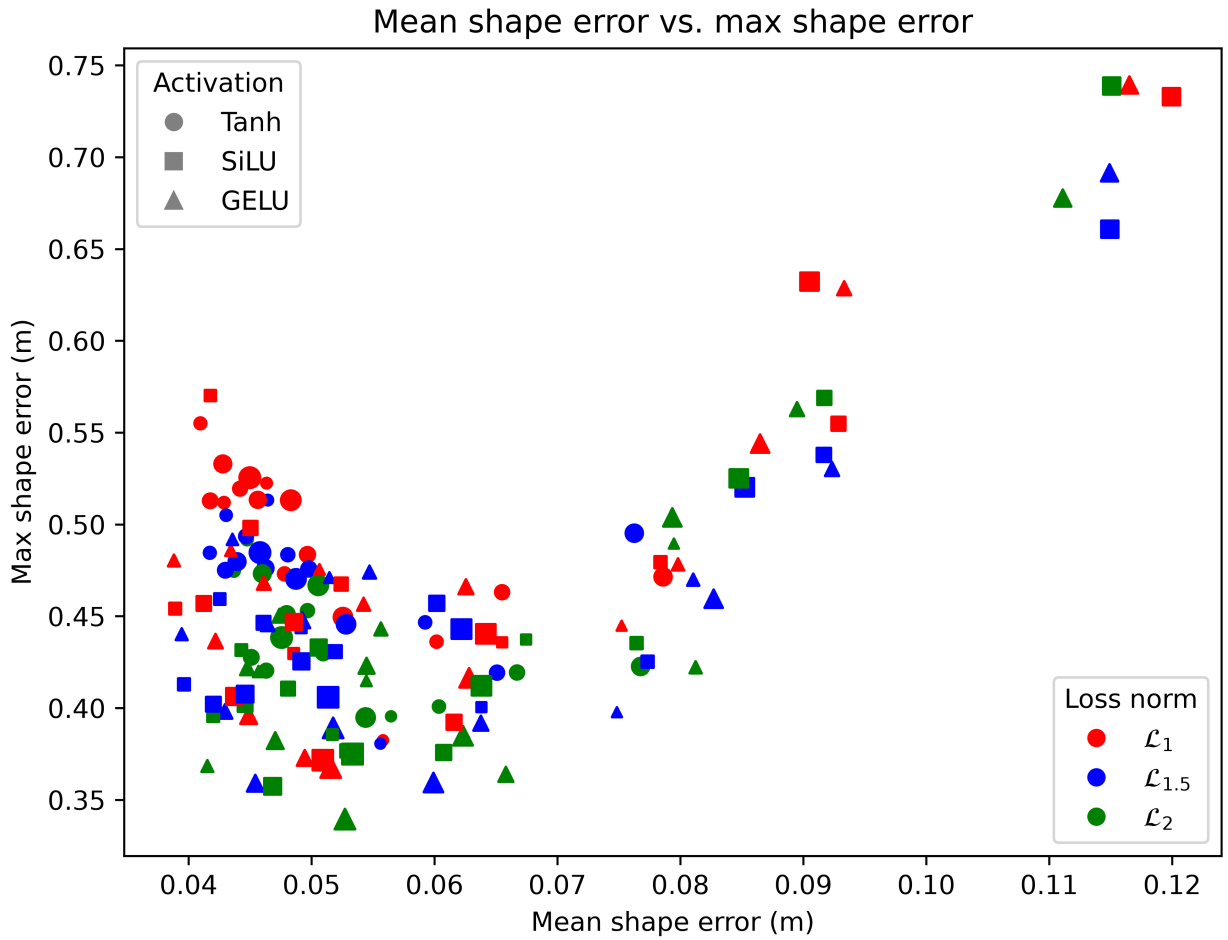

Figure 12: Mean vs. max shape error for test models. Shape indicates activation, color indicates the loss norm used for training, and size indicates model size (number of layers times $\log_2$ of width).

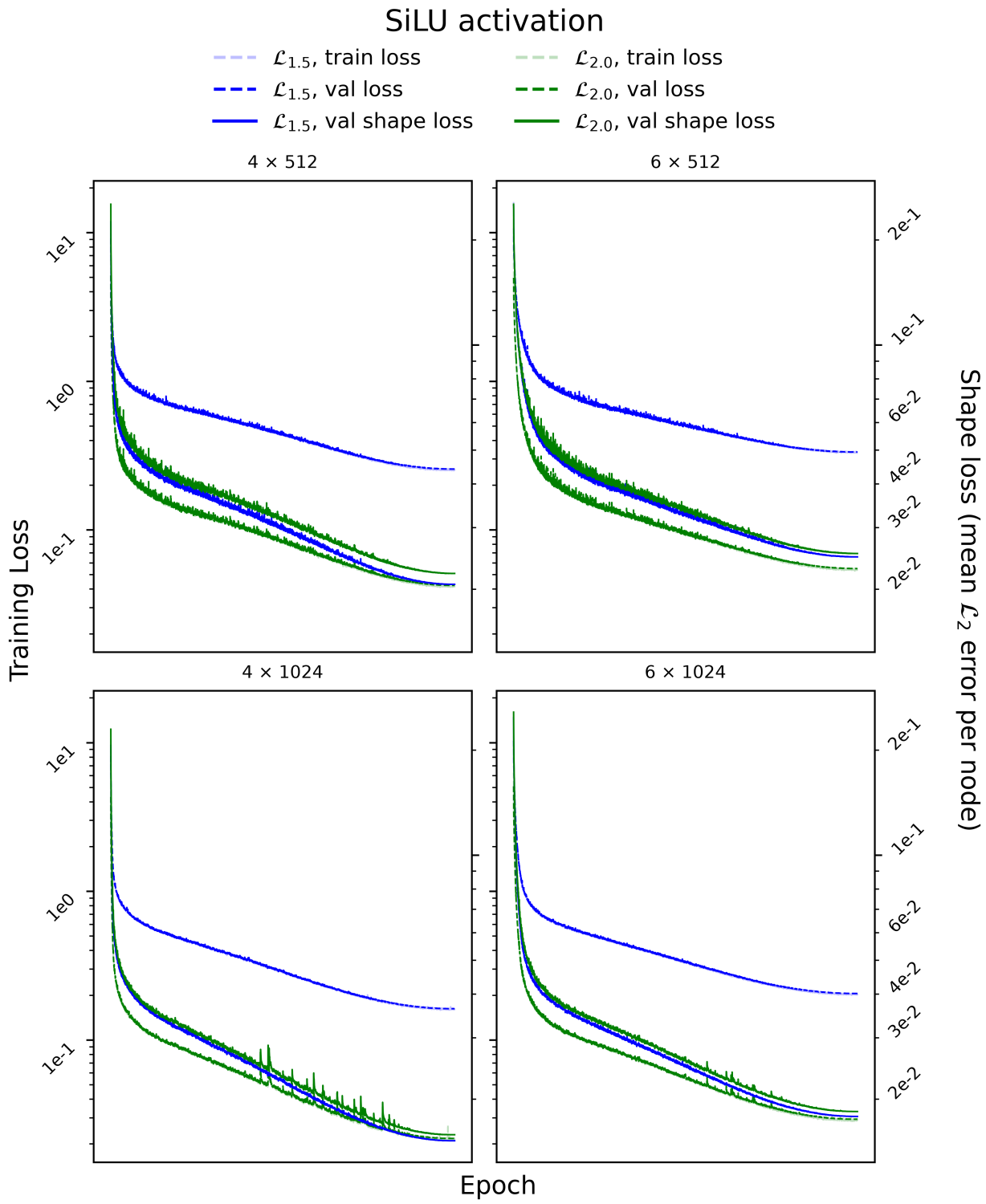

Figure 13: Second-round SiLU model performance.

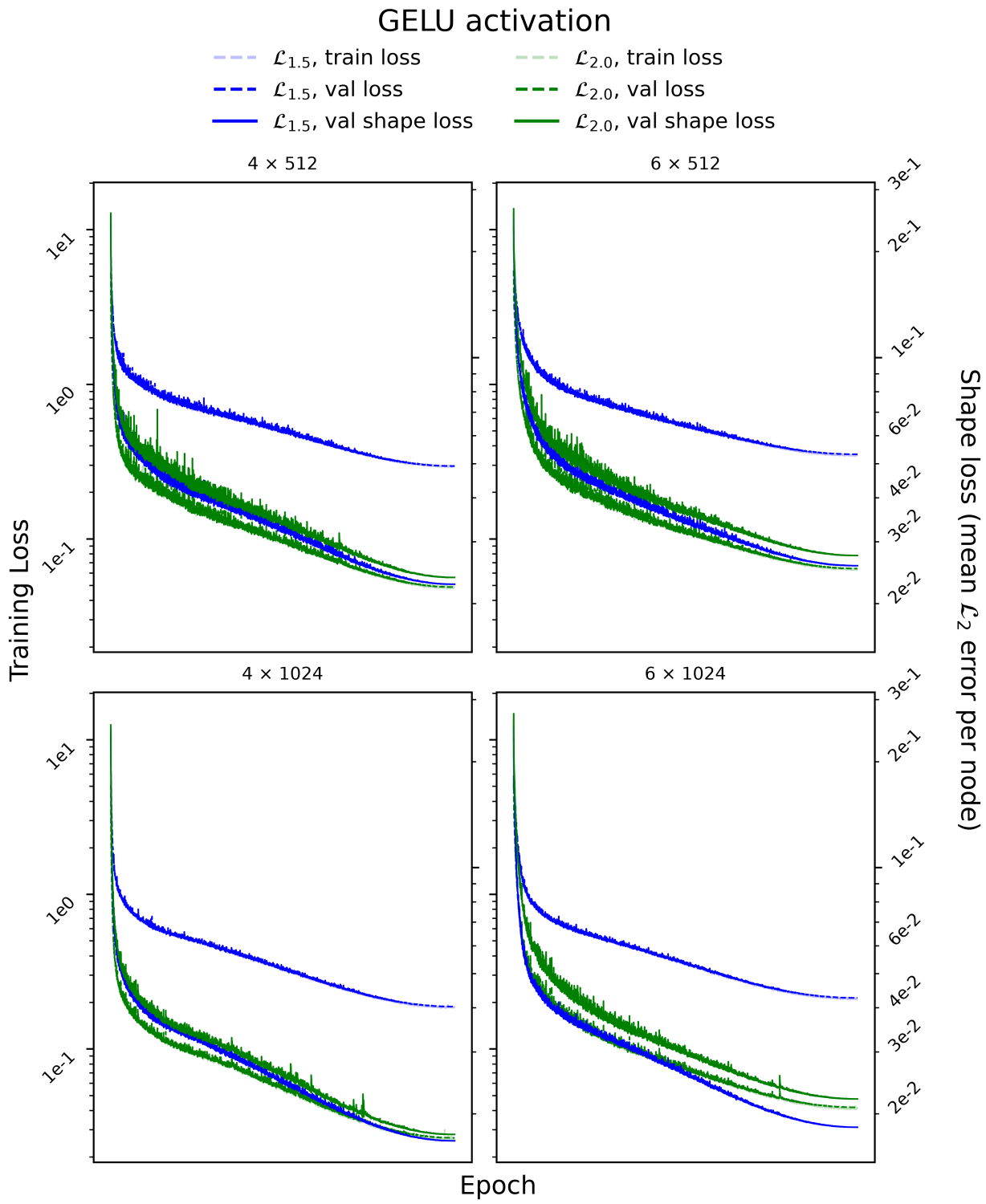

Figure 14: Second-round GELU model performance.

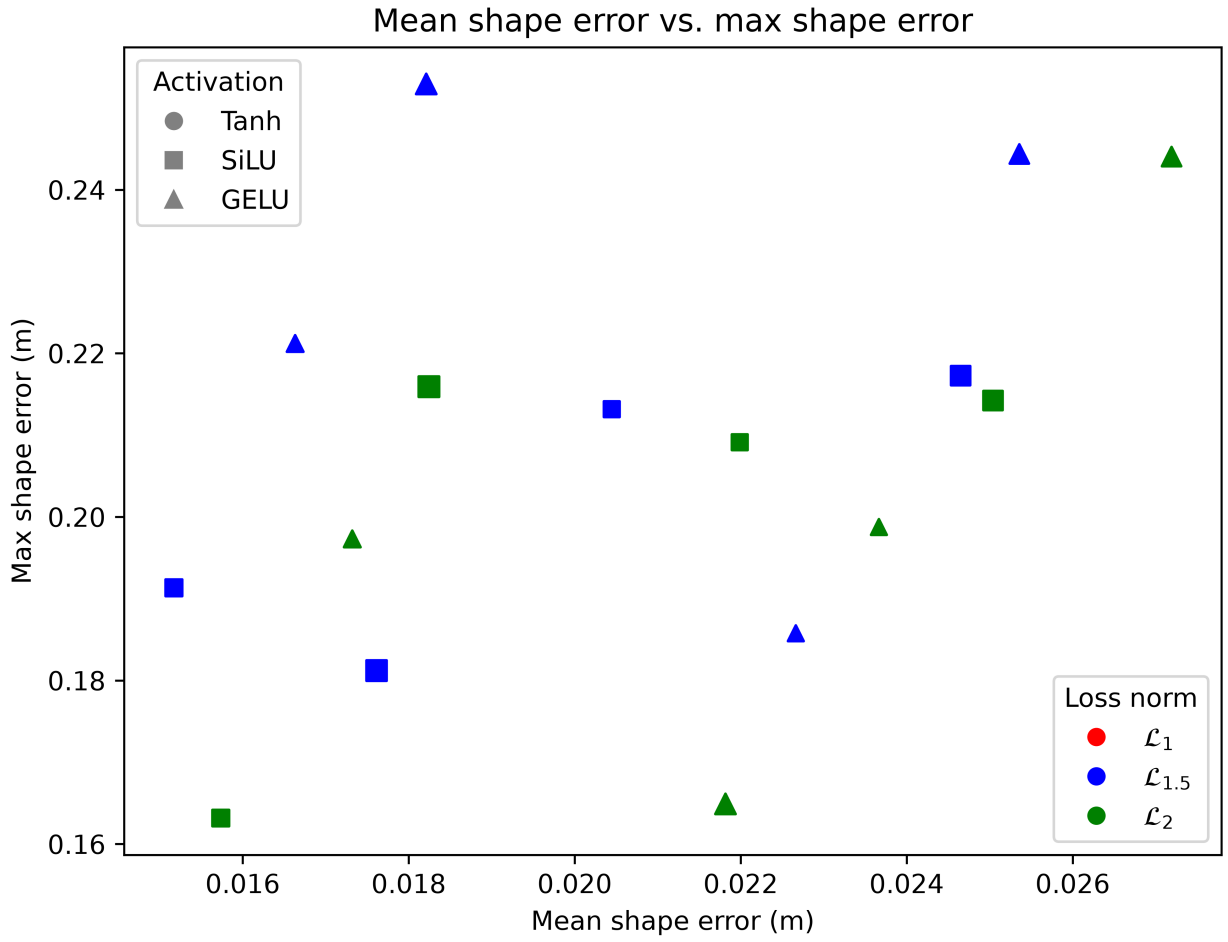

Figure 15: Mean vs. max shape error for second-round test models.

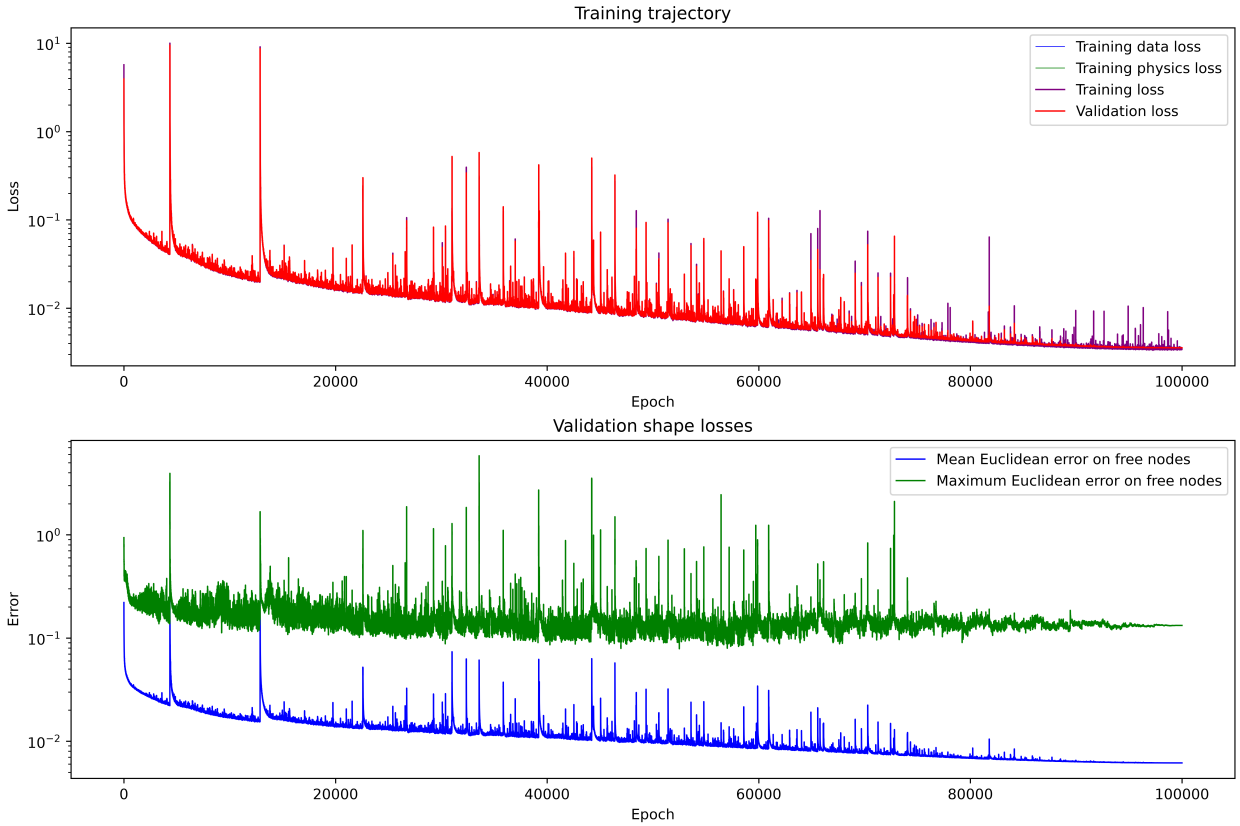

Figure 16: Training trajectory for the final 4×1024 SiLU model, trained for 100,000 epochs.

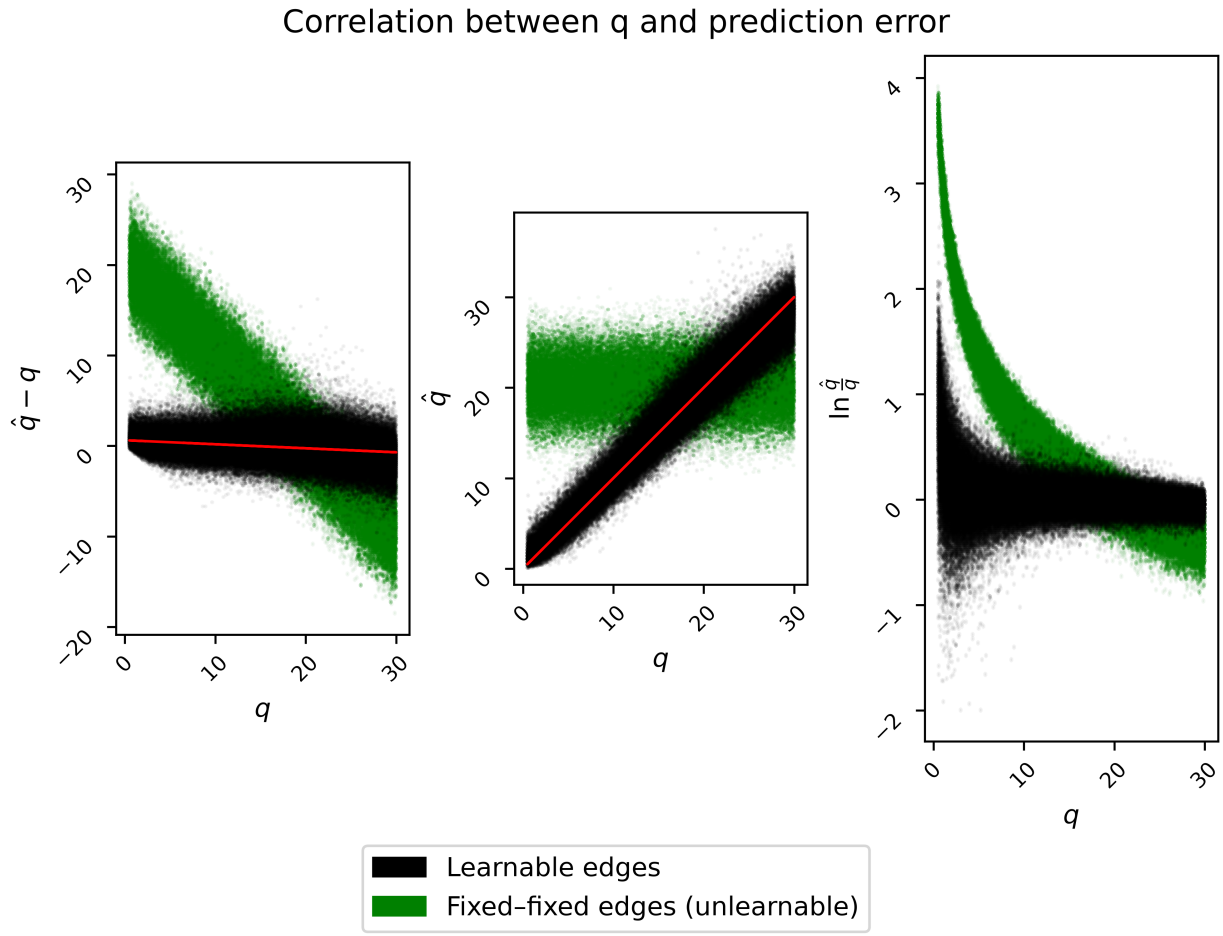

Figure 17: Correlation between force density and prediction error for the final model.

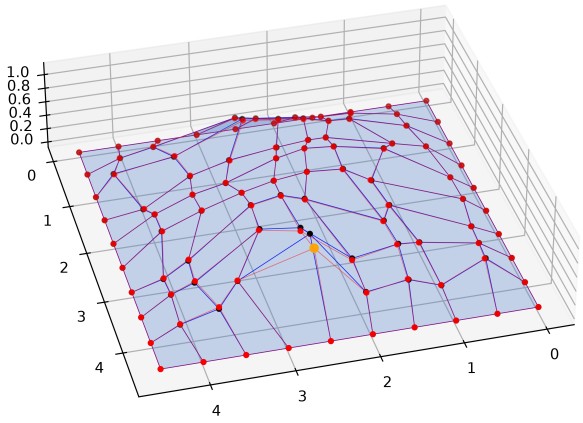

Figure 18: Performance of the final model on the worst sample from the test set. The error is found in an area with low force densities—I believe I can conclude that low force-density areas are more difficult to learn due to ill-conditioning.

epochs), it seems that with an indefinitely long training period, the mean nodal error would truly approach zero. I believe the same would occur for the maximum nodal error, though it would take much longer to train due to the ill-conditioning of the problem. Ill-conditioning is discussed a bit more in section 3.2.2. Perhaps the largest unanswered question I am left with at the end of this project is whether it is possible to reformulate the problem in a way that is faster to learn, or if this is truly a fundamental aspect of the force density operator.

3.2 Next steps

This project greatly restricted the problem space for feasibility, but the natural next step would be to expand the space again to better reflect the type of problems designers actually want to solve. This section describes some potential next steps that would better represent the problem space of interest and hopefully improve the ability of future models.

3.2.1 Improving the training data

There are at least two steps I could take to improve the training data.

I selected force densities and loadings from a uniform random distribution, where the force densities and loadings of adjacent nodes and edges are independent. However, in real-world situations, force and loading are more likely to be spatially-correlated fields. Selecting training data from a smoother noise field would likely make the data easier for the network to learn because it could allow adjacent edges to influence each other. Whether this is the desired result, I think, depends on whether I prioritize more accuracy, or that the model has actually learned the static equilibrium operator.

As stated earlier, I trained solely on funicular targets because it allows a constant baseline—zero loss is theoretically

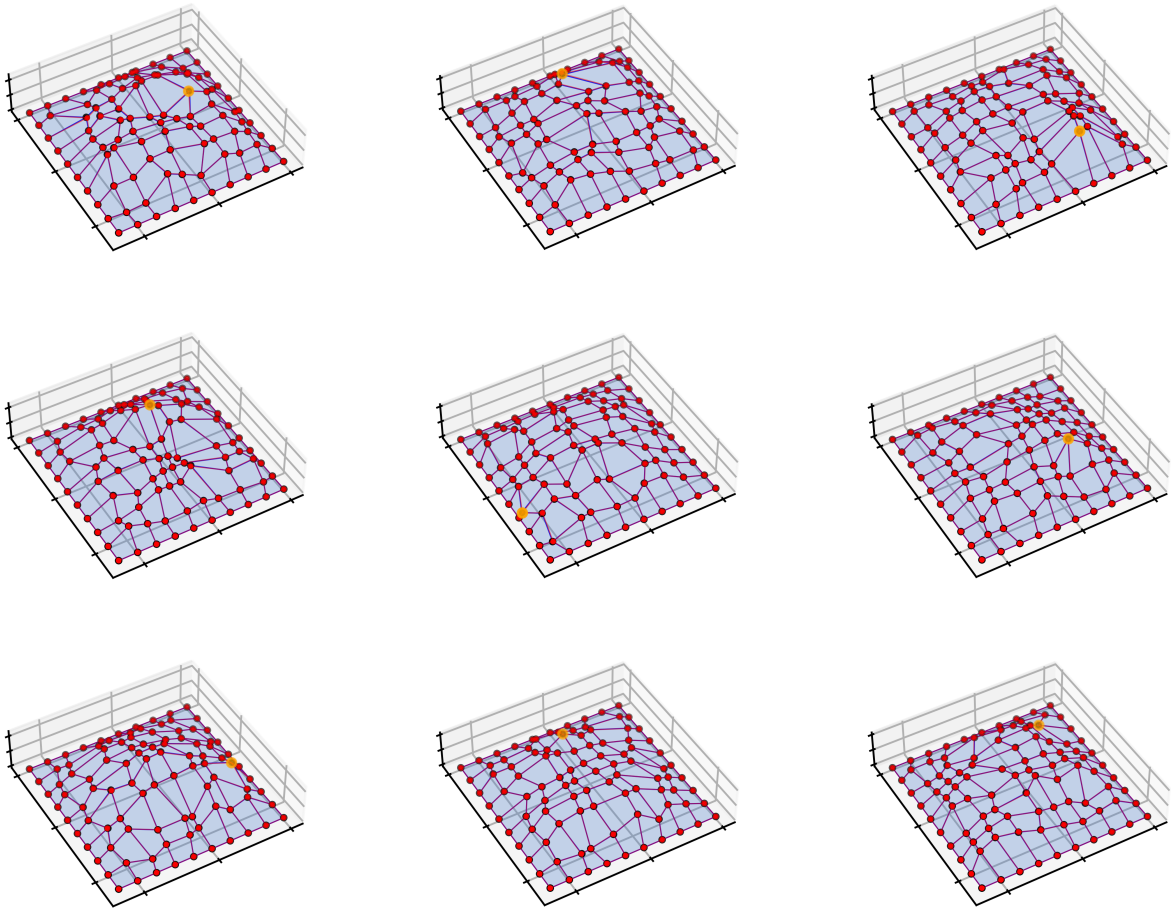

Figure 19: Performance of the final model on a grid of samples from the test set.

possible for a network that has learned the operator. However, in real use-cases, the target geometries are often non-funicular, and the minimum shape loss is unknown. It would be interesting to see what results my current setup yields when trained on non-funicular targets.

For this project, I resisted training the model on the raw force densities because I thought it would be less interesting, but it may also have yielded better results.

3.2.2 A different loss

For this project, I tested out two losses: a shape loss based on geometric distance between target points and predicted points, and a physics loss measuring the force imbalance at target points under the predicted forces.

The two losses have different conditioning. The physics residual is linear in $\tilde{q}$, making the physics loss quadratic in $\tilde{q}$. Equation (3) could be rewritten as

$$\boldsymbol{r}(\boldsymbol{X};\hat{\boldsymbol{Q}}) = \boldsymbol{C}_n^T\tilde{\boldsymbol{Q}}\boldsymbol{C}\boldsymbol{X} = \boldsymbol{A}\tilde{q}$$

where $\boldsymbol{A}$ is the edge-vector matrix of the system (each row containing an edge direction vector) and $\tilde{q}$ is the force-density error. Then the condition number of the Hessian is $\kappa(\boldsymbol{A}^T\boldsymbol{A}) = \kappa(\boldsymbol{A})^2$. (This should be taken over learnable edges because the $\boldsymbol{A}^T\boldsymbol{A}$ is singular if fixed-fixed edges are included.) The shape loss is similar, but has an extra factor. Let $\boldsymbol{Q}$ be the true force densities, and let $\hat{\boldsymbol{Q}} = \boldsymbol{Q} + \tilde{\boldsymbol{Q}}$ be the predicted force densities. Both of them satisfy equilibrium—one for the target shape, and one for the predicted shape $\hat{\boldsymbol{X}} = \boldsymbol{X} + \tilde{\boldsymbol{X}}$. Then

$$\boldsymbol{C}_n^T\boldsymbol{Q}\boldsymbol{C}\boldsymbol{X} = \boldsymbol{C}_n^T(\boldsymbol{Q} + \tilde{\boldsymbol{Q}})\boldsymbol{C}(\boldsymbol{X} + \tilde{\boldsymbol{X}}).$$

The shape error $\tilde{X}$ is the solution of

$$-\boldsymbol{C}_n^T\tilde{\boldsymbol{Q}}\boldsymbol{C}\boldsymbol{X} = \boldsymbol{C}_n^T(\boldsymbol{Q} + \tilde{\boldsymbol{Q}})\boldsymbol{C}\tilde{\boldsymbol{X}}.$$

Since the error on the fixed nodes is zero, this is equivalent to

$$-\boldsymbol{C}_n^T\tilde{\boldsymbol{Q}}\boldsymbol{C}\boldsymbol{X} = \boldsymbol{C}_n^T(\boldsymbol{Q} + \tilde{\boldsymbol{Q}})\boldsymbol{C}_{\boldsymbol{n}}\tilde{\boldsymbol{X}},$$

essentially multiplying the physics residual by the pseudo-inverse of the weighted graph Laplacian $\boldsymbol{D}_n$. $\boldsymbol{D}_n$ is ill-conditioned because the force densities span more than an order of magnitude of stiffness scales. Also, eigenvalues are small for large-scale shape modes and large for high-frequency shape details. The condition number of the Hessian is bounded by $\kappa(\boldsymbol{H}) \leq \kappa(\boldsymbol{D}_n)^2\kappa(\boldsymbol{A})^2$. So, the physics loss is better conditioned, but it more indirectly measures the actual goal (shape accuracy). Maybe using an energy loss

$$\mathcal{L}_{energy} = \tilde{\boldsymbol{X}}^T\boldsymbol{D}_{\boldsymbol{n}}\tilde{\boldsymbol{X}}$$

would split the difference and lead to better results.

I also briefly tried predicting $\log q$ rather than q, since this would help alleviate the span of more than an order of magnitude. I tried adding a loss term for the maximum nodal shape error, since it stubbornly refuses to decrease even as the mean shape error goes down. For my brief trials of these two interventions, the training results were

worse, but I think they are worth revisiting.

3.2.3 Variable fixities

I trained all of the models so far on the same topology—a 10 by 10 grid of nodes, fixed at the exterior boundary. However, I also passed in fixity information with the nodes as part of their feature vectors. The same network setup would potentially be able to learn structures with varying assignments of fixed nodes.

3.2.4 Mixed-mode structures

Mixed-mode structures pose a problem because the shape becomes more sensitive to changes in q as the force density approaches zero. The closer we allow force densities to get to zero, the worse the conditioning of the problem is, the lower the learning rate must be, and the slower the model can learn. Additionally, even without zero force densities appearing, cancellation of tension and compression can make the matrix solve singular. So it seems in addition to bounding the minimum force density, we would need to keep track of the condition number of the system matrix during training. Maybe we could add a loss term specifically to try to keep the system matrix well-conditioned.

On the other hand, removing zero force densities means the model will never be possible to learn one-way structures, such as barrel vaults, which only carry force in one direction. Methods such as thrust-network analysis are already proficient for these structures, but it is a little unfortunate to cut them out entirely.

3.2.5 Variable topologies

My ultimate goal would be to train a general model that can predict force densities for arbitrary topologies and loadings, including changing connectivities and numbers of nodes. I think this would be better suited to a graph neural network. The edge embedding would be decoded to predict force densities. However, since equilibrium is a global (rather than local) property, there would potentially need to be a relatively involved global state in order to learn the force density operator.

3.2.6 Other thoughts

As topologies get more complex, it would also be useful to be able to represent the goal topology and the predicted mesh using different bases. Currently, the goal and predicted topology must be equivalent because the loss is a per-node shape error. A different proximity-based error would allow the goal topology to be represented using a mesh of a different resolution, a NURBs surface, etc.

The force density method is geometrically linear and assumes loads move as nodes move. However, for most structural applications, we apply an area load, and the load on each node would change as the nodes move and their tributary areas change. Previous research I did in undergrad shows that this actually has a significant effect on the equilibrium geometry, so the effect is worth incorporating. Finding the ground truth solution requires iterating through updating loads due to changing tributary areas. Maybe a neural network could learn the nonlinear operator in one go.

References

Pastrana, Rafael et al. *Real-time design of architectural structures with differentiable mechanics and neural networks*. arXiv:2409.02606 [cs.CE]. Mar. 2025. DOI: 10.48550/arXiv.2409.02606. URL: http://arxiv.org/abs/2409.02606 (visited on 10/22/2026).

Schek, H. -J. "The force density method for form finding and computation of general networks". In: *Computer Methods in Applied Mechanics and Engineering* 3.1 (Jan. 1974), pp. 115–134. ISSN: 0045-7825. DOI: 10.1016/0045-7825(74)90045-0. URL: https://www.sciencedirect.com/science/article/pii/0045782574900450 (visited on 10/23/2026).